Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 24 · Part E: Machine Learning for Signals Level 4

Track: Autoencoders, PCA Equivalence, Denoising, Reconstruction-Error Anomaly Detection, Contamination, VAE

Prerequisites: Problems 21, 22, 23 · **Estimated effort:** 16–22 hours

Autoencoders: Learning What Normal Looks Like

An autoencoder compresses and reconstructs its input; trained on ordinary data only, it reconstructs ordinary data well and anomalies poorly. Prove numerically that a linear autoencoder is PCA, build convolutional and denoising versions, detect a burst in a light curve and odd spectra from reconstruction error, and run the experiments that show when the method silently fails: contaminated training data and a bottleneck that is too wide.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/24_autoencoders_for_denoising_and_anomalies/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random

seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

In an anomaly-detection setting you rarely have labelled anomalies, but you often have plenty of "normal" data: months of quiet telemetry, thousands of ordinary stellar spectra, hours of burst-free background. An autoencoder learns to squeeze its input through a narrow bottleneck and expand it again. If the bottleneck is narrow enough, it can only represent patterns typical of the training data, and an input outside that family is reconstructed badly: the reconstruction error is the anomaly score. This is the idea behind the final stage of the gamma-ray-burst pipeline in your session notebook, which slides a 64-sample window along the normalised light curve and flags windows with large error.

The same architecture trained with a noisy input and a clean target is a denoiser. Both uses carry traps: a bottleneck that is too wide copies everything, including anomalies; training data contaminated with the anomalies teaches the network to reconstruct them; and a threshold chosen on the wrong data produces false alarms by the thousand. These traps are the substance of this problem.

Theoretical Framework

1. Autoencoder

An encoder $\boldsymbol{z} = f_\theta(\boldsymbol{x}) \in \mathbb{R}^k$ and a decoder $\hat{\boldsymbol{x}} = g_\phi(\boldsymbol{z})$ are trained to minimise the reconstruction loss $\mathcal{L} = \frac{1}{N}\sum_i \|\boldsymbol{x}_i - g_\phi(f_\theta(\boldsymbol{x}_i))\|^2$. With a bottleneck $k < d$ the network learns a k-dimensional description. A *denoising* autoencoder is trained on pairs (corrupted input, clean target); a *convolutional* autoencoder uses convolution, pooling and up-sampling (or transposed convolution) layers and exploits translation structure.

2. Linear Autoencoder and PCA

For a linear autoencoder (f and g linear, no activation) with mean-centred data, the loss is minimised when the decoder spans the subspace of the k leading principal components, and the minimum equals the PCA reconstruction error $\sum_{i>k}\lambda_i$ (sum of the discarded covariance eigenvalues). Gradient descent does not return the principal axes themselves, only the same *subspace*; compare subspaces with principal angles.

3. Reconstruction-Error Anomaly Score

For a window $\boldsymbol{x}$ the score is $s(\boldsymbol{x}) = \frac{1}{d}\,\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$. Choose the threshold from the distribution of scores on *held-out normal data*, for example its 99.9th percentile, or from the false-alarm requirement. Because scores of an anomaly depend on its scale, normalise each window (subtract the window median, divide by a robust scale or by the square root of the local mean for Poisson data).

4. Failure Modes

(i) *Contamination:* if anomalies are present in the training data, the network learns to reconstruct them. (ii) *Excess capacity:* with $k \geq d$ (or a highly expressive nonlinear decoder) the identity map is a solution and all inputs, anomalous or not, are reconstructed perfectly. (iii) *Pointless sensitivity:* small distribution shifts in normal data (a drifting background) raise the error of normal windows.

5. Variational Autoencoder (bonus)

A VAE encodes to a distribution $q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}, \operatorname{diag}\boldsymbol{\sigma}^2)$ and maximises the evidence lower bound $\mathbb{E}_q[\ln p_\theta(\boldsymbol{x}|\boldsymbol{z})] - \mathrm{KL}\big(q_\phi(\boldsymbol{z}|\boldsymbol{x})\,\|\,\mathcal{N}(0,\mathbf{I})\big)$ using the reparametrisation $\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$.

Data and Simulation Specification

Spectra	`sp_common.synth.stellar_spectra(n_normal=2000, n_anomalous=40, n_bins=256)`: train on normal spectra only; test on 500 normal plus all anomalies; anomaly types: emission line, broad trough, cosmic-ray spikes.
Light curve	`sp_common.synth.grb_event_list`, binned at 0.5 s; 200 burst-free light curves for training (different seeds, `amp=0`), test curves with injected `amp`$= 10, 30, 100, 300$; windows of 64 bins with stride 8; normalise each window with Anscombe transform and median removal.
Denoising data	Windows of 128 samples containing a Gaussian pulse or a transit-like dip at random position and depth, with known clean target, noise levels $\sigma = 0.1, 0.2, 0.5$.
Framework	PyTorch on CPU; fixed seeds; at least three seeds for every reported number.
Real data (strongly recommended)	AstroSat CZTI or Fermi GBM event file (as in problem 22), burst-free stretch for training; or SDSS optical spectra of stars (via the SDSS SkyServer or `astroquery`); or NASA SMAP/MSL spacecraft telemetry anomalies (Hundman et al., KDD 2018; data in the `telemanom` GitHub repository).

Your Task

1. **Linear autoencoder versus PCA (Stage 2).** Train a linear autoencoder with $k = 1, \ldots, 20$ hidden units on the normal spectra. Compare its reconstruction error with PCA's $\sum_{i>k} \lambda_i / d$ for each k and compute the principal angles between the autoencoder decoder subspace and the PCA subspace. Explain the result and what changes when you add a ReLU.
2. **Convolutional autoencoder for spectra.** Build a 1D convolutional autoencoder with a bottleneck of size k (sweep $k = 2, 4, 8, 16, 32, 64$). Plot the distribution of reconstruction errors for normal and anomalous spectra, report ROC AUC, average precision, and recall at a budget of 20 flagged spectra, per anomaly type, and compare with PCA error and Isolation Forest (problem 22).
3. **Denoising autoencoder.** Train on the pulse and dip windows with noise $\sigma = 0.2$. Compare output SNR (`sp_common.stats.snr_db`) and peak-amplitude bias with the Savitzky–Golay

filter, the running median and wavelet shrinkage (problem 18) and with the Wiener filter. Test at noise levels not seen in training.

4. **Burst detection in a light curve.** Train the window autoencoder (architecture of the session notebook: Conv1D with 32 and 16 filters of lengths 5 and 3, pooling, a small bottleneck and the mirrored decoder) on burst-free windows only. Compute window scores for the test curves, set the threshold at the 99.9th percentile of the scores on held-out burst-free windows, map flagged windows back to time, and report efficiency versus injected `amp` and the false-alarm rate on 500 null curves (Wilson intervals). Compare with the n-sigma and DBSCAN detectors of problem 22 at the same false-alarm rate.
5. **Contamination study.** Retrain with 0%, 1%, 5% and 20% of the training windows containing bursts. Plot the efficiency at fixed false-alarm rate against contamination and explain the trend. Try a trimmed loss (ignore the highest-error 5% of windows each epoch) or iterative self-cleaning and measure the recovery.
6. **Capacity and bottleneck failure.** Sweep the bottleneck from 2 to 128 for the window autoencoder and show the ROC AUC collapsing as k approaches the window length; demonstrate that a deep nonlinear decoder can reconstruct the burst perfectly even at a small k when trained long enough, so that the bottleneck alone is not a safeguard. Describe the safeguards you would use (early stopping on normal-only validation error, small capacity, ensembling).
7. **Real data and final verdict.** Apply the detector (autoencoder, plus the baseline n-sigma/DBSCAN detector of problem 22) to the real data. Report candidates with their scores and the null-calibrated false-alarm probabilities, use the agreement of independent methods as a consistency check, and state a verdict in the form "detected, marginal or not detected" with the supporting numbers.

Self-check targets

A linear autoencoder with k hidden units converges to the PCA error $\sum_{i>k} \lambda_i$ (to within optimisation error) and its decoder subspace has principal angles near 0° to the PCA subspace. For $k = d$ the reconstruction error approaches zero for *all* inputs. With a threshold at the 99.9th percentile of normal scores, about 0.1% of normal windows are flagged by construction; verify it on a fresh set.

Aerospace Application Link

Spacecraft produce far more telemetry than can be downlinked, and operators want novelty prioritised. Describe how you would deploy your autoencoder as an onboard novelty detector for housekeeping channels such as temperatures, currents and wheel speeds: the training data (flight-qualified normal operations), the input window, the thresholds and what to do on a flag (increase sampling rate, store a pre-trigger buffer, downlink first). Estimate its memory, computation per window and the number of false alerts per month you can tolerate, and compare it with a simple limit-checking scheme.

Deliverables

D1 **Linear autoencoder versus PCA:** error curves and principal-angle results.

D2 **Spectral anomaly detection:** convolutional autoencoder, score histograms, metrics, per-type recall and comparison with PCA and Isolation Forest.

D3 **Denoising autoencoder:** the comparison table including amplitude bias.

D4 **Burst detection:** the windowed autoencoder pipeline with efficiency-versus-amplitude and

false-alarm statistics against the classical detectors.

D5 **Failure-mode studies and real data:** contamination and capacity experiments, the real-data candidate table and the verdict.

D6 **Memo (300 words):** "Can we trust an unsupervised novelty detector on a spacecraft?" Cite at least two external sources.

Bonus (a) Train a variational autoencoder, use the negative ELBO as the anomaly score and compare with the plain autoencoder. (b) Add a latent-space Mahalanobis score. (c) Build the detector in streaming mode with a one-window delay and measure the trigger latency.

Learning Resources

Autoencoders

- Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*, Ch. 14 (autoencoders).
- Hinton, G.E., Salakhutdinov, R.R. (2006). "Reducing the dimensionality of data with neural networks." Science 313, 504.
- Vincent, P., Larochelle, H., Bengio, Y., Manzagol, P.-A. (2008). "Extracting and composing robust features with denoising autoencoders." Proc. ICML, 1096.
- Kingma, D.P., Welling, M. (2014). "Auto-encoding variational Bayes." arXiv:1312.6114.

Anomaly Detection in Science and Spacecraft

- Hundman, K., Constantinou, V., Laporte, C., Colwell, I., Soderstrom, T. (2018). "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding." Proc. KDD, 387, arXiv:1802.04431.
- Baron, D. (2019). "Machine learning in astronomy: a practical overview." arXiv:1904.07248.
- Chandola, V., Banerjee, A., Kumar, V. (2009). "Anomaly detection: a survey." ACM Computing Surveys 41, 15.

Data and Tools

- SDSS SkyServer: `https://skyserver.sdss.org`; telemanom (SMAP/MSL data and code): `https://github.com/khundman/telemanom`
- PyTorch: `https://pytorch.org/docs/stable/`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.